CS24301 — THEORY (3.0 CR) | 10-PAGE MASTER REVISION | BIT MESRA | NEP SCHEME

Compiler Design — 10-Page Master Quick Revision

High-Yield Formula Sheet, Decision Trees & Top 10 BIT Mesra PYQ Solutions

⚡ 10-Page Master Quick Revision — Compiler Design (CS24301)

⚡ Master Formula, Algorithm & Grammar Cheat Sheet

Topic / Concept	Core Mathematical Formulation / Rule	Key Exam Insight
Immediate Left Recursion	$A \to A\alpha \mid \beta \implies A \to \beta A', \quad A' \to \alpha A' \mid \epsilon$	Eliminates infinite loops in top-down parsers.
Left Factoring	$A \to \alpha\beta_1 \mid \alpha\beta_2 \implies A \to \alpha A', \quad A' \to \beta_1 \mid \beta_2$	Defers decision until lookahead is clear.
2D Array (Row-Major)	$\text{Addr}(A[i_1, i_2]) = \text{base} + \Big((i_1 - l_1)n_2 + (i_2 - l_2)\Big) \times w$	Standard in C / C++ / Java.
2D Array (Column-Major)	$\text{Addr}(A[i_1, i_2]) = \text{base} + \Big((i_2 - l_2)n_1 + (i_1 - l_1)\Big) \times w$	Standard in FORTRAN / MATLAB.
Reaching Definitions	$\text{OUT}[B] = \text{GEN}[B] \cup (\text{IN}[B] \setminus \text{KILL}[B])$	Forward data-flow; confluence by $\cup$.
Available Expressions	$\text{OUT}[B] = e\_\text{GEN}[B] \cup (\text{IN}[B] \setminus e\_\text{KILL}[B])$	Forward data-flow; confluence by $\cap$.
Live Variables	$\text{IN}[B] = \text{USE}[B] \cup (\text{OUT}[B] \setminus \text{DEF}[B])$	Backward data-flow; confluence by $\cup$.

🔥 Top 10 High-Yield BIT Mesra Exam Questions & Answers

Q1. Trace the 6 phases of a compiler for the statement `a = b + c * 50`. (10 Marks)

1. **Lexical:** `id1 = id2 + id3 * 50`
2. **Syntax:** Parse tree with `*` subtree evaluated before `+`
3. **Semantic:** Type conversion `inttofloat(50)` injected
4. **ICG (TAC):** `t1 = inttofloat(50); t2 = id3 * t1; t3 = id2 + t2; id1 = t3`
5. **Optimization:** `t1 = id3 * 50.0; id1 = id2 + t1`
6. **Code Gen:** Emits target machine assembly with CPU registers.

Q2. Compare LL(1), SLR(1), CLR(1), and LALR(1) parsers. (8 Marks)

- **LL(1):** Top-down predictive parser using 1 lookahead; cannot handle left recursive grammars.
- **SLR(1):** Simple LR bottom-up parser using LR(0) items; places reduce actions in $\text{FOLLOW}(A)$.
- **CLR(1):** Canonical LR parser using LR(1) items with specific lookaheads; largest number of states.
- **LALR(1):** Merges CLR(1) states having identical LR(0) cores; same number of states as SLR(1) with near CLR(1) power.